

Laporan Praktikum Minggu Ke-5 Kontrol Cerdas

Minggu ke-5

Nama : Satrio Sri Gunawan
NIM : 224308069
Kelas : TKA 7C
Akun Github (Tautan) : <https://github.com/satriosrigunawan>

1. Judul Percobaan

Deep Reinforcement Learning untuk Kontrol Kompleks

2. Tujuan Percobaan

Tujuan dari percobaan pada praktikum kali ini, sebagai berikut:

1. Memahami konsep dasar kontrol cerdas (intelligent control systems).
2. Memahami konsep Deep Reinforcement Learning (DRL) dalam kontrol sistem kompleks.
3. Mengimplementasikan Deep Q-Network (DQN) untuk kontrol otomatis.
4. Menganalisis performa DRL dibandingkan dengan metode kontrol konvensional.

3. Landasan Teori

Deep Reinforcement Learning (DRL) adalah pengembangan dari Reinforcement Learning (RL) yang menggunakan jaringan saraf untuk mempelajari tindakan terbaik dalam lingkungan yang kompleks. Dalam RL, agen belajar dengan mencoba berbagai aksi dan menerima reward dari lingkungan untuk menemukan strategi terbaik. Namun, RL tradisional sulit diterapkan pada sistem dengan data besar dan kontinu. Dengan bantuan deep learning, DRL mampu mengenali pola dan memperkirakan nilai tindakan (Q-value) secara lebih efisien (Arulkumaran et al., 2017).

Salah satu algoritma populer dalam DRL adalah Deep Q-Network (DQN), yang menggantikan tabel Q klasik dengan jaringan saraf untuk memperkirakan nilai Q dari setiap aksi. DQN menggunakan experience replay agar pelatihan lebih stabil dan epsilon-greedy untuk menyeimbangkan eksplorasi serta eksploitasi (Kapturowski et al., 2018). Algoritma ini banyak digunakan pada sistem robotika, kendaraan otonom, dan pengendalian energi karena dapat beradaptasi dengan perubahan lingkungan tanpa memerlukan model sistem yang rumit (Li, 2018).

4. Diskusi

1. Kelebihan dan Kelemahan DQN dibandingkan Metode Q-Learning Klasik

Deep Q-Network (DQN) memiliki keunggulan dibandingkan Q-Learning klasik karena mampu menangani sistem yang kompleks dan ruang keadaan yang besar. Jika Q-Learning tradisional menggunakan tabel Q untuk menyimpan nilai setiap keadaan dan aksi, DQN menggantinya dengan jaringan saraf yang dapat memperkirakan nilai Q secara langsung. Hal ini membuat DQN lebih efisien dan mampu bekerja pada lingkungan dinamis serta berdimensi tinggi. Selain itu, mekanisme *experience replay* membantu menjaga kestabilan pelatihan dengan mengurangi korelasi data, sedangkan strategi *epsilon-greedy* menjaga keseimbangan antara eksplorasi dan eksploitasi.

Meski demikian, DQN juga memiliki kekurangan. Algoritma ini membutuhkan sumber daya komputasi besar dan waktu pelatihan lebih lama karena melibatkan jaringan saraf yang kompleks. Kinerjanya sangat bergantung pada pengaturan parameter seperti *learning rate* dan *discount factor*. Jika tidak dikonfigurasi dengan tepat, DQN dapat mengalami ketidakstabilan atau *overfitting*. Oleh karena itu, meskipun DQN lebih canggih, penerapannya memerlukan perancangan dan keahlian teknis yang baik.

2. Adaptasi DQN untuk Kontrol Sistem Dinamis seperti Drone atau Robot Industri

Untuk menerapkan DQN pada sistem dinamis seperti drone atau robot industri, diperlukan penyesuaian agar algoritma dapat beradaptasi dengan lingkungan yang selalu berubah. Tahap awal meliputi pendefinisian *state* dan *action*

secara tepat, misalnya posisi, orientasi, dan kecepatan sebagai *state*, serta perubahan sudut baling-baling atau gaya dorong sebagai *action*. Data dari sensor seperti IMU, kamera, dan encoder digunakan agar agen DQN memperoleh informasi sistem secara real time.

Proses pelatihan biasanya dilakukan di lingkungan simulasi seperti OpenAI Gym, Gazebo, atau PyBullet untuk menghindari risiko kerusakan perangkat nyata. Setelah model stabil, hasilnya dapat diterapkan ke perangkat fisik melalui *transfer learning*. Dalam praktiknya, DQN sering dikombinasikan dengan kontrol konvensional seperti PID atau Fuzzy Logic guna menjaga kestabilan sistem. Keberhasilan pembelajaran juga sangat bergantung pada perancangan *reward function* yang sesuai dengan tujuan, misalnya menjaga keseimbangan atau mencapai target posisi secara efisien. Dengan rancangan yang tepat, DQN mampu beradaptasi secara mandiri dan menghasilkan kontrol yang akurat pada sistem dinamis kompleks.

5. Assignment

Pada tugas minggu kelima ini dikembangkan sistem pengenalan gestur tangan real-time menggunakan kombinasi **Support Vector Machine (SVM)** dan **MediaPipe** yang terhubung dengan **OpenCV**. Proses diawali dengan memuat dataset titik kunci tangan menggunakan `pd.read_csv()`, kemudian menyesuaikan nama kolom agar konsisten. Label diambil dari kolom pertama, sedangkan fitur (X) berasal dari kolom kedua hingga sebelum kolom terakhir. Data dibagi menjadi 80% untuk pelatihan dan 20% untuk pengujian menggunakan `train_test_split()`.

Model SVM dengan kernel linear dilatih menggunakan `SVC(kernel='linear')` dan diuji untuk menghitung akurasi melalui `accuracy_score()`. Setelah model terlatih, sistem diterapkan secara real-time dengan MediaPipe dan OpenCV. Kamera menangkap citra, mengubahnya ke format RGB, lalu mendeteksi titik-titik tangan menggunakan `hands.process()`. Jika tangan terdeteksi, koordinat x, y, dan z setiap titik digunakan sebagai input ke model SVM untuk memprediksi jenis gestur. Hasil prediksi kemudian ditampilkan di layar dengan `cv2.putText()`.

Dengan menggabungkan MediaPipe untuk pelacakan titik kunci tangan dan SVM untuk klasifikasi gestur, sistem ini mampu mengenali posisi serta gerakan tangan secara akurat dalam waktu nyata, menunjukkan penerapan efektif antara

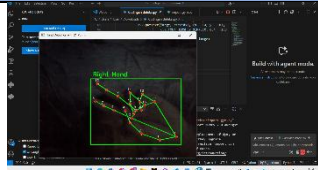
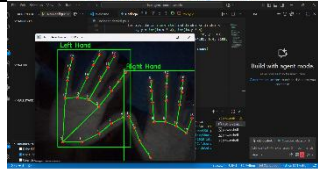
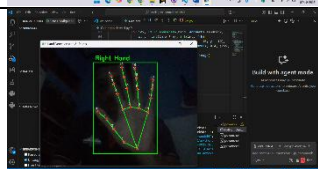
machine learning dan *computer vision* dalam analisis gerakan manusia. Berikut ini adalah fungsi dari kode program yang digunakan seperti pada tabel berikut ini :

No	Bagian Kode / Fungsi	Penjelasan
1	<code>import cv2</code>	Mengimpor pustaka OpenCV untuk menangani pengolahan citra dan akses kamera.
2	<code>import mediapipe as mp</code>	Mengimpor pustaka MediaPipe untuk melakukan deteksi dan pelacakan tangan secara real-time.
3	<code>mp_hands = mp.solutions.hands</code>	Menginisialisasi modul Hands dari MediaPipe yang berfungsi mendeteksi dan melacak titik tangan (landmarks).
4	<code>mp_drawing = mp.solutions.drawing_utils</code>	Menginisialisasi fungsi utilitas untuk menggambar hasil deteksi landmark di atas frame video.
5	<code>cap = cv2.VideoCapture(0)</code>	Membuka koneksi ke kamera (0 berarti kamera bawaan laptop). Jika ada kamera eksternal, indeks bisa diganti (misal 1 atau 2).
6	<code>with mp_hands.Hands(...) as hands:</code>	Membuat objek deteksi tangan dengan parameter: - <code>max_num_hands=2</code> : mendeteksi maksimal dua tangan. - <code>min_detection_confidence=0.7</code> : batas minimal kepercayaan deteksi. - <code>min_tracking_confidence=0.7</code> : batas minimal kepercayaan pelacakan.
7	<code>ret, frame = cap.read()</code>	Membaca satu frame dari kamera. <code>ret</code> bernilai <i>True</i> jika pembacaan berhasil.
8	<code>cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)</code>	Mengubah format warna dari BGR (OpenCV) ke RGB (MediaPipe membutuhkan format RGB).
9	<code>results = hands.process(rgb_frame)</code>	Memproses frame RGB untuk mendeteksi tangan dan menghasilkan koordinat titik-titik (landmarks).
10	<code>if results.multi_hand_landmarks:</code>	Mengecek apakah ada tangan yang terdeteksi pada frame.
11	<code>for hand_landmarks in results.multi_hand_landmarks:</code>	Melakukan iterasi untuk setiap tangan yang terdeteksi (bisa satu atau dua).
12	<code>mp_drawing.draw_landmarks(...)</code>	Menggambar titik-titik sendi dan koneksi antar sendi tangan di atas

		frame video dengan warna dan ketebalan tertentu.
13	<code>cv2.imshow("Deteksi Gerakan Tangan - Titik Sendi", frame)</code>	Menampilkan hasil deteksi tangan secara real-time di jendela OpenCV.
14	<code>if cv2.waitKey(1) & 0xFF == ord('q'):</code>	Memberi opsi keluar dari program jika tombol 'q' ditekan di keyboard.
15	<code>cap.release()</code>	Menutup koneksi kamera setelah program berhenti.
16	<code>cv2.destroyAllWindows()</code>	Menutup semua jendela tampilan OpenCV.

6. Data dan Output Hasil Pengamatan

Tabel berikut menunjukkan hasil pengamatan deteksi gerakan tangan menggunakan sistem berbasis MediaPipe. Setiap gerakan dianalisis berdasarkan jumlah tangan yang terdeteksi, jumlah titik sendi per tangan, posisi atau titik penting pada tangan, serta keterangan visualisasi output yang dihasilkan.

No	Dokumentasi	Deteksi	Jumlah Titik Sendi per Tangan	Keterangan Output Visual
1		Gerakan Tangan membentuk 2 jari. Terdeteksi Satu tangan	21 titik	Garis hijau mengikuti bentuk jari, visualisasi sendi tetap akurat
2		Gerakan Tangan membentuk 10 jari Terdeteksi 2 Tangan	42 titik	Garis hijau menghubungkan sendi, titik merah di setiap sendi
3		Gerakan Tangan terbuka normal Jumlah Tangan Terdeteksi 1	21 titik	Garis hijau menghubungkan sendi, titik merah di setiap sendi

7. Kesimpulan

Berdasarkan hasil praktikum dan analisis yang dilakukan, dapat disimpulkan bahwa program yang dibuat berhasil mengklasifikasikan gestur tangan secara real-time menggunakan kombinasi model **SVM** dan **MediaPipe**. Akurasi model sangat dipengaruhi oleh kualitas dataset dan pengaturan parameter, sehingga hasil deteksi dapat ditingkatkan melalui penyempurnaan data, penyesuaian model, serta optimasi performa sistem. Secara umum, sistem mampu mengenali gerakan tangan dengan baik, meskipun masih mengalami kendala akibat *noise* dan pencahayaan yang tidak stabil. Selain itu, sistem hanya dapat mendeteksi hingga dua tangan sekaligus dan belum mampu mengenali objek lain di luar tangan pengguna.

8. Saran

Berdasarkan hasil praktikum dan analisis, beberapa saran perbaikan berikut dapat dilakukan untuk meningkatkan akurasi dan performa sistem.

1. **Perluasan Dataset:** Menambah jumlah dan variasi data agar mencakup beragam kondisi pencahayaan serta posisi tangan, sehingga model menjadi lebih adaptif dan akurat.
2. **Optimasi Model dan Kinerja Real-Time:** Melakukan penyetelan *hyperparameter* pada SVM (seperti C dan γ), memperbaiki efisiensi kode, melakukan kompresi citra, serta menambahkan proses *post-processing* seperti *time smoothing* agar hasil prediksi lebih stabil.
3. **Peningkatan Kapasitas Deteksi:** Mengembangkan sistem agar mampu mengenali lebih dari dua tangan sekaligus maupun objek lain yang relevan.

9. Daftar Pustaka

- Lugaresi, C., Tang, J., Nash, W., McClanahan, C., Uboweja, E., Hays, M., ... & Grundmann, M. (2019).
- Arulkumaran, K., Deisenroth, M. P., Brundage, M., & Bharath, A. A. (2017). *Deep Reinforcement Learning: A Brief Survey*.